

Spaniel

Pseudo-objects and interfaces in Typst

API reference · version 0.0.1

Contents

1	Preface	2
2	Public API	2
2.1	Types	2
2.2	Runtime objects	5
2.3	Operations & implementations	7
2.4	Extensions & worlds	9
2.5	Dispatch	10
3	Internal API	13
3.1	Results	13
3.2	Type internals	14
3.3	Pattern matching & specificity	15
3.4	Registry internals	17
3.5	Dispatch internals	18
4	Appendix	21
4.1	Example scope	21

1 Preface

Please note that this document contains examples that include undocumented functions and variables. These exist only to support the examples and are not part of the public API. They are defined Section 4.1, which describes the scope used when evaluating the examples.

In brief, these terms are as follows: `Int`, `int`, `Vector`, `vector_of`, `vector`, `Mul`, `mul_int_int`, `S`, `T`, `U`, `mul_scalar_vector`, `algebra`, `world`, and `mul`.

2 Public API

2.1 Types

Type expressions: the vocabulary used to describe runtime types.

Defines nominal types, type constructors, and type variables, along with the structural operations the dispatcher relies on — equality, concreteness checks, variable collection, substitution, keying, and display.

apply_type

Apply a type constructor to type arguments.

```
#display_type(apply_type(Vector, Int))
```

```
Vector[Int]
```

Parameters

```
apply_type(  
  constructor: dictionary ,  
  ..arguments: arguments  
) -> dictionary
```

constructor dictionary

The type constructor to apply.

..arguments arguments

Type-expression arguments; the count must equal the constructor's arity.

display_type

Render a type expression as a human-readable string.

```
#display_type(vector_of(Int))
```

```
Vector[Int]
```

Parameters

```
display_type(ty: dictionary) -> str
```

ty dictionary

The type expression to render.

nominal_type

Construct a nominal runtime type.

validate checks the unboxed payload stored in an object of this type.

```
#let Int = nominal_type("demo/Int", name:
  "Int", validate: v => type(v) == int)
#Int.name
```

Int

Parameters

```
nominal_type(
  id: str,
  name: str | none,
  validate: function
) -> dictionary
```

id str

Stable, globally unique identifier for the type.

name str or none

Human-readable display name; defaults to id.

Default: none

validate function

Validator for the unboxed payload of objects of this type.

Default: value => true

substitute_type

Substitute type variables in ty according to bindings.

Variables absent from bindings are left unchanged.

```
#display_type(substitute_type(vector_of(T),
  ("T": Int)))
```

Vector[Int]

Parameters

```
substitute_type(
  ty: dictionary,
  bindings: dictionary
) -> dictionary
```

ty dictionary

The type expression to rewrite.

bindings dictionary

Mapping from variable name to replacement type expression.

type_constructor

Construct an n-ary type constructor, such as `Vector[_]`.

Its validator receives `(type_arguments, payload)`.

```
#let Vector = type_constructor("demo/Vector",  
1, name: "Vector")  
#Vector.arity
```

1

Parameters

```
type_constructor(  
  id: str,  
  arity: int,  
  name: str or none,  
  validate: function  
) -> dictionary
```

id str

Stable, globally unique identifier for the constructor.

arity int

Number of type arguments the constructor takes.

name str or none

Human-readable display name; defaults to id.

Default: none

validate function

Validator receiving `(type_arguments, payload)`.

Default: `(args, value) => true`

type_equal

Test whether two type expressions are structurally equal.

Ignores validators and display names.

```
#repr((type_equal(Int, Int), type_equal(Int,  
vector_of(Int))))
```

(true, false)

Parameters

```
type_equal(  
  lhs: dictionary,  
  rhs: dictionary  
) -> bool
```

lhs dictionary

First type expression.

rhs dictionary

Second type expression.

type_key

A stable key for a type expression or pattern.

This intentionally ignores validators and display names.

```
#type_key(vector_of(Int))
```

```
apply(demo/Vector;nominal(demo/Int))
```

Parameters

```
type_key(ty: dictionary) -> str
```

ty dictionary

The type expression to key.

type_variable

Construct a type variable for use in implementation patterns.

```
#repr(type_variable("T"))
```

```
(kind: "type.variable", name: "T")
```

Parameters

```
type_variable(name: str) -> dictionary
```

name str

The variable's name, e.g. "T".

2.2 Runtime objects

Runtime objects: values boxed together with their runtime type.

Provides the boxing constructor and the predicates that let the dispatcher recognise objects, read their type, and validate their payloads.

is_object

Test whether a value is a protocol object.

```
#repr((is_object(object(Int, 42)),  
is_object(42)))
```

```
(true, false)
```

Parameters

```
is_object(value: any) -> bool
```

value any

The value to test.

object

Box an unboxed Typst value with a user-defined runtime type.

```
#object(Int, 42).value
```

```
42
```

Parameters

```
object(  
  ty: dictionary,  
  value: any  
) -> dictionary
```

ty dictionary

The concrete runtime type to attach.

value any

The unboxed value to box.

payload_valid

Validate an unboxed value against a concrete runtime type.

```
#repr((payload_valid(Int, 42),  
payload_valid(Int, "oops")))
```

```
(true, false)
```

Parameters

```
payload_valid(  
  ty: dictionary,  
  value: any  
) -> bool
```

ty `dictionary`

The concrete type to validate against.

value `any`

The unboxed value to check.

runtime_type

Return the runtime type of a protocol object.

```
#display_type(runtime_type(object(Int, 42)))
```

Int

Parameters

`runtime_type(value: dictionary) -> dictionary`

value `dictionary`

The protocol object to inspect.

2.3 Operations & implementations

Operations, requirements, and implementations.

The declarations an author writes to describe a generic operation, the implementations that satisfy it for particular type patterns, and the requirements one implementation may place on others.

implementation

Declare an implementation of an operation.

The body is called as `body(context, ..objects)`.

```
#let mul-ii = implementation(  
  Mul, (Int, Int), Int,  
  (ctx, lhs, rhs) => object(Int, lhs.value *  
    rhs.value),  
  name: "Int × Int -> Int",  
)  
#mul-ii.name
```

Int × Int -> Int

Parameters

```
implementation(  
  operation: dictionary,  
  inputs: array,  
  output: dictionary,  
  body: function,  
  constraints: array,  
  name: str none,  
  priority: int  
) -> dictionary
```

operation dictionary

The operation being implemented.

inputs array

Input type patterns, one per argument; may contain type variables.

output dictionary

Output type expression produced by the implementation.

body function

Implementation body, called as `body(context, ..objects)`.

constraints array

Requirements (see [requires\(\)](#)) that must hold for this implementation to apply.

Default: `()`

name str or none

Human-readable display name; defaults to the operation's name.

Default: `none`

priority int

Tie-breaking priority among equally specific implementations; higher wins.

Default: `0`

operation

Declare an operation: a named, fixed-arity dispatch point that implementations can be registered against.

```
#let Mul = operation("demo/Mul.mul", 2, name:
"mul")
#Mul.arity
```

Parameters

```
operation(
  id: str,
  arity: int,
  name: str none
) -> dictionary
```

id `str`

Stable, globally unique identifier for the operation.

arity `int`

Number of object arguments the operation accepts.

name `str` or `none`

Human-readable display name; defaults to id.

Default: `none`

requires

Require another operation to be resolvable while checking an implementation.

inputs is an array of type expressions. If output is supplied, matching the required operation's output against it may introduce new bindings.

```
#repr(requires(Mul, (S, T), output:
U).inputs.map(display_type))
```

```
("S", "T")
```

Parameters

```
requires(
  operation: dictionary,
  inputs: array,
  output: dictionary none
) -> dictionary
```

operation `dictionary`

The operation that must be resolvable.

inputs `array`

Input type expressions for the required call; must match operation's arity.

output `dictionary` or `none`

Optional output type expression to match the required operation's output against.

Default: `none`

2.4 Extensions & worlds

Extensions and world building.

Bundles operations and implementations into extensions, then validates and merges them into an immutable dispatch world — checking arities, resolving references, rejecting duplicates, and ensuring every type variable is bound.

build_world

Combine extensions into a validated immutable dispatch world.

```
#let w = build_world(algebra)
#w.implementations.len()
```

2

Parameters

`build_world(..extensions: arguments) -> dictionary`

..extensions `arguments`

The extensions to combine, each produced by `extension()`.

extension

Bundle operations and implementations into an extension for `build_world()`.

```
#extension(operations: (Mul,),
implementations:
(mul_int_int,)).operations.len()
```

1

Parameters

`extension(
 operations: array,
 implementations: array
) -> dictionary`

operations `array`

Operations declared by this extension.

Default: ()

implementations `array`

Implementations provided by this extension.

Default: ()

2.5 Dispatch

Multiple dispatch.

Resolves an operation against a world by selecting the single most-specific implementation for the argument types — recursively satisfying constraints and guarding against cycles — then invokes it and revalidates its result.

dispatch

Invoke an operation using multiple dispatch.

```
#repr(dispatch(world, Mul, int(2),
vector(Int, (1, 2, 3))).value)
```

(2, 4, 6)

Parameters

```
dispatch(  
  world: dictionary,  
  operation: dictionary,  
  ..arguments: arguments  
) -> dictionary
```

world dictionary

The dispatch world to resolve against.

operation dictionary

The operation to invoke.

..arguments arguments

The object arguments to dispatch on.

dispatcher

Produce a convenient operation-specific function.

```
#let mul = dispatcher(world, Mul)  
#mul(int(6), int(7)).value
```

42

Parameters

```
dispatcher(  
  world: dictionary,  
  operation: dictionary  
) -> function
```

world dictionary

The dispatch world to bind.

operation dictionary

The operation to bind.

try_resolve

Resolve an operation for concrete type descriptors without invoking it.

```
#display_type(try_resolve(world, Mul, (Int,  
vector_of(Int))).value.output)
```

Vector[Int]

Parameters

```
try_resolve(  
  world: dictionary ,  
  operation: dictionary ,  
  actual_types: array  
) -> dictionary
```

world dictionary

The dispatch world to resolve against.

operation dictionary

The operation to resolve.

actual_types array

Concrete type descriptors for each argument.

3 Internal API

The definitions below are private, underscore-prefixed helpers. They are **not** part of the public API and may change without notice; they are documented here for contributors working on the package internals.

3.1 Results

Internal result type.

Lightweight ok/err values used to thread fallible computations — type matching, constraint solving, and resolution — without panicking, so that callers can report rich diagnostics instead.

`_err`

Wrap a message as a failed result.

```
#repr(_err("no matching implementation"))
```

```
(  
  kind: "result.err",  
  message: "no matching implementation",  
)
```

Parameters

```
_err(message: str) -> dictionary
```

message str

The error message.

`_is_ok`

Test whether a result is successful.

```
#repr((_is_ok(_ok(1)), _is_ok(_err("boom"))))
```

```
(true, false)
```

Parameters

```
_is_ok(result: dictionary) -> bool
```

result dictionary

The result to test.

`_ok`

Wrap a value as a successful result.

```
#repr(_ok(42))
```

```
(kind: "result.ok", value: 42)
```

Parameters

```
_ok(value: any) -> dictionary
```

value `any`

The success payload.

3.2 Type internals

Type expressions: the vocabulary used to describe runtime types.

Defines nominal types, type constructors, and type variables, along with the structural operations the dispatcher relies on — equality, concreteness checks, variable collection, substitution, keying, and display.

`_is_concrete_type`

Test whether a type expression is fully concrete, i.e. contains no type variables or rigid variables.

```
#repr((_is_concrete_type(vector_of(Int)),  
_is_concrete_type(vector_of(T))))
```

```
(true, false)
```

Parameters

```
_is_concrete_type(ty: dictionary) -> bool
```

ty `dictionary`

The type expression to test.

`_is_type_expression`

Test whether a value is a well-formed type expression: a nominal type, type variable, valid constructor application, or internal rigid variable.

```
#repr((_is_type_expression(Int),  
_is_type_expression(42)))
```

```
(true, false)
```

Parameters

```
_is_type_expression(value: any) -> bool
```

value `any`

The value to test.

`_type_variables`

Collect the names of all type variables occurring in ty.

```
#repr(_type_variables(apply_type(Vector, T)))
```

```
("T",)
```

Parameters

```
_type_variables(ty: dictionary) -> array
```

ty dictionary

The type expression to scan.

3.3 Pattern matching & specificity

Pattern matching and specificity.

Matches type patterns against concrete types (accumulating type-variable bindings) and decides when one implementation's signature is strictly more specific than another's, which drives overload selection in the dispatcher.

_match_signature

Match a whole signature: each pattern against the corresponding actual type, threading one shared set of bindings across all positions.

```
#let b = _match_signature((S, vector_of(T)),  
  (Int, vector_of(Int))).value  
#(display_type(b.at("S")) + ", " +  
  display_type(b.at("T")))
```

Int, Int

Parameters

```
_match_signature(  
  patterns: array,  
  actual_types: array  
) -> dictionary
```

patterns array

The input type patterns of an implementation.

actual_types array

The concrete argument types of a call.

_match_type

Match a single type pattern against an actual type, extending bindings with any type variables resolved along the way.

```
#let matched = _match_type(T, Int, (:))  
#display_type(matched.value.at("T"))
```

Int

Parameters

```
_match_type(  
  pattern: dictionary,  
  actual: dictionary,  
  bindings: dictionary  
) -> dictionary
```

pattern dictionary

The type pattern to match; may contain type variables.

actual dictionary

The concrete type to match against.

bindings dictionary

Type-variable bindings accumulated so far.

`_more_specific`

Test whether implementation lhs is strictly more specific than rhs: rhs accepts every call lhs does, but not vice versa.

```
#let generic = implementation(Mul, (S, T),  
Int, (ctx, l, r) => l, name: "generic")  
#let specific = implementation(Mul, (Int,  
Int), Int, (ctx, l, r) => l, name:  
"specific")  
#repr((_more_specific(specific, generic),  
_more_specific(generic, specific)))
```

(true, false)

Parameters

```
_more_specific(  
  lhs: dictionary,  
  rhs: dictionary  
) -> bool
```

lhs dictionary

The implementation being tested for greater specificity.

rhs dictionary

The implementation to compare against.

`_signature_subsumes`

Test whether general subsumes specific: every call described by specific is also described by general. The variables in specific are made rigid (via prefix) so only general may generalise over them.

```
#repr((  
  _signature_subsumes((S,), (Int,), "a"),  
  _signature_subsumes((Int,), (S,), "b"),  
))
```

(true, false)

Parameters

```
_signature_subsumes(  
  general: array,  
  specific: array,  
  prefix: str  
) -> bool
```

general array

The candidate more-general signature (its variables stay flexible).

specific array

The candidate more-specific signature (its variables are made rigid).

prefix str

A prefix that makes the rigidified variables unique.

`_skolemize_type`

Replace every type variable in `ty` with a fresh rigid variable, tagged by `prefix`, so it matches only itself. Used to make a signature's variables opaque while testing subsumption.

```
#display_type(_skolemize_type(vector_of(T),  
  "impl"))
```

Vector[<T>]

Parameters

```
_skolemize_type(  
  ty: dictionary,  
  prefix: str  
) -> dictionary
```

ty dictionary

The type expression to rigidify.

prefix str

A prefix that makes the generated rigid variables unique.

3.4 Registry internals

Extensions and world building.

Bundles operations and implementations into extensions, then validates and merges them into an immutable dispatch world — checking arities, resolving references, rejecting duplicates, and ensuring every type variable is bound.

`_constraint_key`

Build a stable string key identifying a constraint, used to detect duplicate implementations.

```
#_constraint_key(requires(Mul, (Int, Int)))
```

```
requires(demo/Mul.mul;nominal(demo/Int);nominal(demo/Int);none)
```

Parameters

```
_constraint_key(constraint: dictionary) -> str
```

constraint dictionary

The constraint to key.

_implementation_key

Build a stable string key identifying an implementation by its operation, input and output types, constraints, and priority.

```
#_implementation_key(mul_int_int)
```

```
demo/Mul.mul(nominal(demo/Int),nominal(demo/Int))->nominal(demo/Int) where @0
```

Parameters

```
_implementation_key(implementation: dictionary) -> str
```

implementation dictionary

The implementation to key.

_validate_implementation_variables

Assert that every type variable used by an implementation's constraints and output is bound by its input patterns (or by an earlier constraint output), panicking with a descriptive message otherwise.

```
{#  
  _validate_implementation_variables(mul_scalar_ve  
    "validated: every type variable is bound"  
}  
}
```

```
validated: every type variable is bound
```

Parameters

```
_validate_implementation_variables(implementation: dictionary) -> none
```

implementation dictionary

The implementation to check.

3.5 Dispatch internals

Multiple dispatch.

Resolves an operation against a world by selecting the single most-specific implementation for the argument types — recursively satisfying constraints and guarding against cycles — then invokes it and revalidates its result.

`_call_key`

Build a stable string key for a call site — an operation plus its concrete argument types — used to detect cyclic requirement resolution.

```
#_call_key(Mul, (Int, Int))
```

```
demo/Mul.mul(nominal(demo/Int),nominal(demo/Int))
```

Parameters

```
_call_key(  
  operation: dictionary,  
  actual_types: array  
) -> str
```

operation dictionary

The operation being called.

actual_types array

The concrete argument types of the call.

`_resolve`

Core resolution routine: find the single most-specific implementation of operation for actual_types, recursively satisfying each implementation's constraints and guarding against cycles via stack. Returns an ok result carrying the chosen candidate, or an err describing why resolution failed.

```
#let resolved = _resolve(world, Mul, (Int,  
Int), ())  
#resolved.value.implementation.name
```

```
Int × Int -> Int
```

Parameters

```
_resolve(  
  world: dictionary,  
  operation: dictionary,  
  actual_types: array,  
  stack: array  
) -> dictionary
```

world dictionary

The dispatch world to resolve against.

operation dictionary

The operation to resolve.

actual_types array

The concrete argument types.

stack array

Call keys currently being resolved, used for cycle detection.

4 Appendix

4.1 Example scope

```
1  #import "/src/lib.typ": *
2  #import "/src/lib.typ" as m-lib
3  #import "/src/internal.typ" as m-internal
4  #import "/src/types.typ" as m-types
5  #import "/src/matching.typ" as m-matching
6  #import "/src/registry.typ" as m-registry
7  #import "/src/dispatch.typ" as m-dispatch
8
9  #let Int = nominal_type(
10   "demo/Int",
11   name: "Int",
12   validate: value => type(value) == int,
13 )
14 #let integer = value => object(Int, value)
15
16 #let Vector = type_constructor(
17   "demo/Vector",
18   1,
19   name: "Vector",
20   validate: (arguments, value) => (
21     type(value) == array
22     and value.all(element => payload_valid(arguments.first(), element))
23   ),
24 )
25 #let vector_of = element_type => apply_type(Vector, element_type)
26 #let vector = (element_type, values) => object(vector_of(element_type), values)
27
28 #let Mul = operation("demo/Mul.mul", 2, name: "mul")
29
30 #let mul_int_int = implementation(
31   Mul,
32   (Int, Int),
33   Int,
34   (ctx, lhs, rhs) => object(Int, lhs.value * rhs.value),
35   name: "Int × Int -> Int",
36 )
37
38 #let S = type_variable("S")
39 #let T = type_variable("T")
40 #let U = type_variable("U")
41
42 #let mul_scalar_vector = implementation(
43   Mul,
44   (S, vector_of(T)),
45   vector_of(U),
46   (ctx, lhs, rhs) => vector(
```

```

47     ctx.bindings.at("U"),
48     rhs.value.map(value => (
49       (ctx.dispatch)(Mul, lhs, object(ctx.bindings.at("T"), value)).value
50     )),
51   ),
52   constraints: (requires(Mul, (S, T), output: U),),
53   name: "S × Vector[T] -> Vector[U]",
54 )
55
56 #let algebra = extension(
57   operations: (Mul,),
58   implementations: (mul_int_int, mul_scalar_vector),
59 )
60 #let world = build_world(algebra)
61 #let mul = dispatcher(world, Mul)
62
63 #let example-scope = (
64   dictionary(m-lib)
65   + dictionary(m-internal)
66   + dictionary(m-types)
67   + dictionary(m-matching)
68   + dictionary(m-registry)
69   + dictionary(m-dispatch)
70   + (
71     Int: Int,
72     int: integer,
73     Vector: Vector,
74     vector_of: vector_of,
75     vector: vector,
76     Mul: Mul,
77     mul_int_int: mul_int_int,
78     S: S,
79     T: T,
80     U: U,
81     mul_scalar_vector: mul_scalar_vector,
82     algebra: algebra,
83     world: world,
84     mul: mul,
85   )

```